NEOBILD

How I Built Local AI on a 300€ Phone

I Had No Choice

Guide Version 2.0 — April 2026

TrinityCore v1.2.0 · Qwen2.5-Coder-1.5B · MNN Chat · Termux

Written entirely on a Xiaomi Redmi Note 14 Pro+
Snapdragon 7s Gen 3 · Android 14 · HyperOS

23	67	8.1K	v1.2.0
GitHub Stars	Cloners (14d)	LinkedIn Impressions	TrinityCore

Contents

Prologue: Why I Had No Choice

After several seizures, my laptop was gone — stolen while I was still recovering. No PC, no budget for a new one, and a brain that didn't work the way it used to.

What I had: a Xiaomi Redmi Note 14 Pro+. 300€. Snapdragon 7s Gen 3. 8GB RAM.

Most people would have stopped. I started.

Not because I had a plan. Because I had no other option — and because I refused to depend on Google, Microsoft, or OpenAI just to use my own computer.

What followed was a long fight against Android limits, the Phantom Process Killer, SIGILL crashes, and quantization bugs that cost me entire nights. This guide is what I wish I'd had back then.

> Version 2.0 reflects the current state of the project: TrinityCore v1.2.0 is live, the Obsidian logger is wired up, the community is building on top of it, and the system runs overnight autonomously on a 300€ phone.

Chapter 1: Termux is Not a Toy

Most people who find Termux treat it like a curiosity. A Linux terminal on Android — cool party trick, not much more.

I treated it like my only computer. Because it was.

The first thing you learn when you actually live in Termux is that Android hates you. The OS is designed for apps that sleep, not processes that run. Every background task is a target. Every service you start is one reboot away from disappearing forever.

The second thing you learn is that most tutorials are written by people with a PC as backup. They skip the parts that break. They don't mention that armv8.7a will SIGILL on the efficiency cores of a Snapdragon 7s Gen 3. They don't tell you that the Phantom Process Killer will silently murder your inference server at 3am with no log entry.

What actually works:

Install from F-Droid — not the Play Store version, which is abandoned.

```
pkg update && pkg upgrade pkg install python git cmake ninja termux-setup-storage
```

That's the foundation. Everything else builds on this.

> ■ Before you build anything: your phone will try to kill everything you create. Chapter 2 is about how to fight back.

Chapter 2: The Phantom Process Killer

Android has a feature that sounds reasonable on paper: if a background process uses too much CPU or memory, the OS kills it.

In practice, it kills everything. Your inference server. Your Python daemons. Your agents that were supposed to run overnight. All of it — silently, without warning, often at the worst possible moment.

This is called the Phantom Process Killer. It was introduced in Android 12 and it is the single biggest obstacle to running local AI on Android.

The multi-layer kill chain:

Mechanism	Trigger	Survival Strategy
Phantom Process Killer	Too many background processes	ADB: max_phantom_processes
Doze Mode	Screen off > 30min	termux-wake-lock
App Standby Buckets	App not in foreground	Battery: Unrestricted
Low Memory Killer	RAM pressure	Keep model ≤ 3B on 8GB
Battery Optimization	Background activity	Disable for Termux

What you can do without root:

Use a watchdog that restarts services within 60 seconds of a kill:

```
while true; do if ! pgrep -f clipboard_daemon.py > /dev/null; then nohup python3 ~/clipboard_daemon.py >> ~/clipboard.log 2>&1 & fi if ! pgrep -f screenshot_watcher.sh > /dev/null; then nohup bash ~/scripts/screenshot_watcher.sh >> ~/watcher.log 2>&1 & fi sleep 60 done &
```

The ADB fix (requires Wireless ADB):

Since Android 11, Wireless ADB works without a PC. Enable under Settings → Developer Options → Wireless Debugging:

```
adb pair 127.0.0.1:<pairing_port> adb connect 127.0.0.1:<adb_port> adb shell device_config put activity_manager max_phantom_processes 2147483647 adb shell settings put global hidden_api_policy 1
```

Note: Some Xiaomi ROMs reset this on reboot. Verify after each restart with: adb shell device_config get activity_manager max_phantom_processes

Chapter 3: Choosing the Right Model

Not every model runs on a phone. Most don't. And the ones that technically run will either crash, hallucinate, or generate at 0.3 tokens per second.

The hard limits:

After Android takes its share, you have roughly 4–4.5GB free on an 8GB device. This rules out anything above 3B parameters. Not "it runs slowly" — it crashes. Every time.

The format that works:

Forget GGUF for now. MNN is the inference engine that actually performs on ARM. Alibaba built it for mobile, and it handles quantization without SIGILL on your efficiency cores. Run as a standalone Android app — not in Termux.

```
# attention_mode formula: # flash_attention x 8 + kv_quant_mode # Mode 14 = TQ4 — recommended for 4B+ models # Mode 6 = standard — safe for all models attention_mode = 14
```

This is not in any tutorial. Found by trial and error after a week of crashes.

Models that actually work:

Model	Size	Speed	Use Case
Qwen2.5-1.5B Q4	~900MB	10–16 tok/s	Daily driver
Qwen2.5-Coder-1.5B Q4	~900MB	10–16 tok/s	Code tasks (current)
SmolLM2-1.7B Q4	~1GB	8–14 tok/s	General use
Qwen2.5-3B Q4	~1.7GB	5–9 tok/s	Complex reasoning

8GB RAM: max 3B parameters · 12GB RAM: up to 4B comfortable

Chapter 4: Building the Stack

A single model running in a terminal is not a stack. It's a demo.

A stack is what happens when you chain things together — inference engine, API layer, automation scripts, agents — so that the whole thing does something useful while you sleep.

The architecture:

```
MNN Chat (port 8080, Android standalone app) | v OpenAI-compatible API | v TrinityCore engine.py (Python agents) | v BLAKE3 hash-chained logs + Obsidian daily log
```

MNN Chat — the inference layer:

MNN Chat runs as a standalone Android app on port 8080. It speaks the OpenAI API format — which means any tool that works with OpenAI will work with your local model. No Termux required, no daemon to manage.

Critical flags:

```
--attention_mode 14 # TQ4 quantization — critical for stability --context_len 4000 # KV cache limit --backend cpu # GPU kernels crash on Adreno — CPU is stable
```

The Python agent layer:

On top of MNN Chat runs TrinityCore — a config-driven multi-agent framework. Full details in Chapter 5.

Secrets management:

```
# Store API keys in ~/.termux_secrets chmod 600 ~/.termux_secrets # Load in Python scripts: with open(os.path.expanduser("~/.termux_secrets")) as f: for line in f: if "=" in line: k, v = line.strip().split("=", 1) os.environ[k] = v
```

Chapter 5: TrinityCore — The Multi-Agent Framework

TrinityCore v1.2.0 is the successor to the original Trinity Orchestrator. Where the original was hardcoded for CVE analysis, TrinityCore is a universal multi-agent framework: config-driven, persona-swappable, deployable for any analytical task.

Architecture:

```
trinity_core/ ■■■ engine.py # Loop engine, BLAKE3 chaining, CISA KEV feed ■■■
config.json # CVE analysis config ■■■ config_reddit.json # Reddit content config ■■■
trinity_ui.py # Flask Web UI on port 5001
```

The four personas:

Persona	Role	Confidence	Function
Dominus	Red Team Researcher	0.2 – 0.4 LOW	Identifies attack vectors
Axiom	Forensic Analyst	0.3 – 0.6 MED	Names concrete IoCs
Cipher	Skeptic	0.5 – 0.8 MED	Challenges prior analysis
Vector	Strategist	0.7 – 0.9 HIGH	Proposes mitigations

Sequential context passing:

Each agent receives only the last sentence of the previous agent's output as context — not the full response. This prevents context bloat on small models while maintaining discourse continuity.

```
# In engine.py: def last_sentence(text): sentences = re.split(r'(?<=[.!?])\s+',
text.strip()) return sentences[-1] if sentences else text[:100] context +=
f"{p['name']}: {last_sentence(answer)}\n"
```

Confidence scoring:

Every agent response gets a confidence score based on hardcoded bounds plus a small bonus for concrete security indicators (CVE IDs, payloads, exploit patterns). Rendered in the UI as color-coded badges: GREEN HIGH / YELLOW MED / RED LOW.

```
AGENT_CONFIDENCE = { "Dominus": (0.2, 0.4), "Axiom": (0.3, 0.6), "Cipher": (0.5, 0.8),
"Vector": (0.7, 0.9), }
```

BLAKE3 hash-chaining:

Every agent response is appended to an immutable chain. Each entry includes the hash of the previous entry — modifying any entry invalidates all subsequent hashes. The chain is verified on startup.

Input sources:

```
"input_source": { "type": "cisa_kev", // Live CISA KEV JSON feed "type": "json_file",
// Custom JSON input "type": "maxxki_json" // MAXXKI code analyzer output (planned) }
```

Quick start:

```
git clone https://github.com/weissmann93/NeoBild cd NeoBild/trinity_core pip install
flask requests blake3 python3 trinity_ui.py # Open http://127.0.0.1:5001
```

Chapter 6: The Obsidian Second Brain

Every process in the stack now logs to an Obsidian vault in real time. This wasn't planned — it emerged from the need to audit what was actually happening while I wasn't watching.

The logger:

~/scripts/termux_logger.py writes daily Markdown files to: /storage/emulated/0/Documents/Meine Gedanken/logs/YYYY-MM-DD.md

```
# Entry format: - `HH:MM:SS` **[Source]** LEVEL: message # CLI usage: python3 ~/scripts/termux_logger.py "Source" "message" "INFO"
```

Sources currently logging:

Source	What it logs
[TrinityCore]	Round start, each agent response (80 chars), engine stop
[WPPoster]	Post published with title
[Termux]	Every bash command via PROMPT_COMMAND hook
[Cron]	Heartbeat every 30min, discourse backup
[ClipboardDaemon]	Start, each clipboard save (chars + filename), stop
[ShotNote]	OCR completion with hash
[ScreenshotWatcher]	Start, each new screenshot detected
[Collector]	Session start/complete, each model response
[AutoTagger]	Run start, tagged/skipped counts
[Boot]	Service start on device reboot
[Watchdog]	Any process restart (WARN level)

The bash hook that logs every Termux command automatically:

```
# In ~/.bashrc: log_command() { local cmd cmd=$(history 1 | sed 's/^[[:space:]]*[0-9]*[[:space:]]//') if [ -n "$cmd" ] && [[ "$cmd" != *"termux_logger"* ]]; then python3 ~/scripts/termux_logger.py "Termux" "$cmd" "CMD" 2>/dev/null & fi } export PROMPT_COMMAND="log_command"
```

Chapter 7: The Crashes I Had So You Don't Have To

Every tutorial shows you the happy path. Nobody writes about the three days they lost to a bug that turned out to be one wrong compiler flag. I'm writing it.

SIGILL on efficiency cores:

The Snapdragon 7s Gen 3 has two CPU core types: A720 (performance) and A520 (efficiency). The A520 cores don't support ARM instructions that the A720 cores do. Compile with -march=armv8.7a or -march=native and your binary will crash silently when Android schedules it on an A520 core.

```
# NEVER use: -march=armv8.7a -march=native # ALWAYS use: -march=armv8-a+dotprod+i8mm
```

OpenCL kernel crashes on Adreno:

MNN supports OpenCL for GPU acceleration. The Adreno 810 technically supports it. The Qwen hybrid linear attention model uses OpenCL kernels that the Adreno 810 rejects. Silent crash. No log.

```
# Fix: CPU-only inference # Slower, but stable. On this chip, stable beats fast.
--backend cpu
```

KV cache memory overflow:

At 4000 token context, the KV cache alone can push you over your memory limit. The process dies without warning. Fix: quantize the KV cache.

```
--cache-type-k q4_0 --cache-type-v q4_0 # Cuts cache memory ~60% with minimal quality
impact
```

The watchdog that wasn't watching:

A watchdog that checks pgrep works fine until the process restarts with a different name or PID. Use port-based health checking instead:

```
while true; do if ! curl -s http://localhost:8080/health > /dev/null; then pkill -f
mnn_chat 2>/dev/null sleep 2 nohup ~/mnn_chat --model ... & fi sleep 60 done &
```

auth failure state caching (OpenClaw):

OpenClaw caches auth failure states for 4 hours. If your API key rotates or you get a 401 error, it will keep failing silently. Fix:

```
rm -rf /tmp/openclaw && pkill -f openclaw
```

Chapter 8: Community & Traction

NeoBild went from a personal project to something people are actually building on. Here's what happened in the first weeks of public release.

GitHub metrics (14 days):

Metric	Value	Context
Total repo views	537	—
Unique visitors	360	—
Git clones	100	Active replication
Unique cloners	**67**	18.6% conversion rate
Stars	23	—
Forks	4	—

18.6% clone-to-visitor conversion is significantly above typical open-source benchmarks (1–5%). This indicates active replication intent, not passive consumption.

Traffic sources:

Reddit Frontpage: 145 views / 95 unique — discovery channel Reddit direct: 76 / 46 — targeted clicks t.co (Twitter/X): 46 / 41 — near 1:1, high intent meet.randorisec.fr: 4 / 3 — organic security community share

The randorisec.fr referral is the most significant: a French security community shared the repo manually without any promotion from my side.

Notable community interactions:

Maximilian Kiefer — Ported TrinityCore to Java, integrated his MAXXKI code analyzer, implemented confidence scoring (0.3 → 0.9) and BLAKE3 VERIFIED status. Building a hybrid version combining static code analysis with LLM discourse.

Snir Balgaly — DevOps Manager at Palo Alto Networks, commented: "Who's this guy. I want his agent."

Shane Hughes — Staff Software Engineer at Splunk/Cisco, connected and engaged with the project.

Carlo Alberto Cacopardo — LLM Engineer at Eons Research (Relational Memory Dynamics), reached out to discuss reasoning drift in multi-turn agent chains.

Chapter 9: What's Next

MAXXKI integration (input mode):

Maximilian's MAXXKI analyzer produces structured JSON with vulnerability findings. TrinityCore will consume this as an input source — replacing the CISA KEV feed with real code analysis results. Dominus, Axiom, Cipher, and Vector will then discuss actual findings from actual codebases.

```
"input_source": { "type": "maxxki_json", "path": "~/maxxki_output.json" }
```

Dynamic confidence scoring:

Current confidence scores are hardcoded per agent with a small keyword bonus. The next version will implement Maximilian's cross-agent agreement bonus: +0.1 when all agents identify the same attack pattern, -0.2 on contradiction.

Prototype Fund application:

Applying for the German Prototype Fund (October 2026 cycle) — up to 47,500€ for open source infrastructure projects. NeoBild qualifies as sovereign AI infrastructure for individuals, with demonstrated community adoption and an 18.6% clone conversion rate as evidence of real-world relevance.

neobildOS:

A sovereign Android AI environment using MNN with TurboQuant TQ3/TQ4 support. GitHub Actions CI for cross-compilation. The goal: a reproducible, installable local AI stack for any Android device.

Epilogue: From a Psychiatric Clinic

I wrote the first version of this guide in a psychiatric clinic in Klingenmünster.

No desk. No second monitor. No proper keyboard. A phone, a lot of time, and the kind of quiet that comes when life forces you to stop.

Most people wait for the right conditions before they build something. Better hardware. More money. More energy. A stable situation.

I didn't have that option. So I built anyway.

What I learned: the constraints don't stop you. They focus you. When you can't install a GPU cluster, you figure out how to make a 1.5B model do the work. When you can't remember what you did yesterday, you build a system that remembers for you. When Android kills your process at 3am, you write a watchdog and go back to sleep.

The phone is not the limitation. The thinking is.

This guide is now version 2.0. The system runs overnight. People are building on it. A Staff Engineer at Palo Alto Networks wants the agent. A developer in Germany ported it to Java the same day he saw it.

I'm still on the same phone. Still in Termux. Still building.

Build something sovereign. Keep your data local. Don't ask permission from companies that don't have your interests at heart.

That's the whole point.

Appendix: Commands, Flags, and References

Essential Termux setup:

```
# Install from F-Droid, not Play Store pkg update && pkg upgrade pkg install python
git cmake ninja clang pkg install termux-api termux-setup-storage
```

MNN Chat — correct startup (in Android app settings):

```
attention_mode: 14 context_len: 4000 backend: cpu port: 8080
```

Compile flags for Snapdragon 7s Gen 3:

```
# NEVER: -march=native or -march=armv8.7a # Both cause SIGILL on A520 efficiency cores
# CORRECT: -march=armv8-a+dotprod+i8mm
```

Boot script (~/.termux/boot/start-services.sh):

```
#!/data/data/com.termux/files/usr/bin/bash termux-wake-lock python3
~/scripts/termux_logger.py "Boot" "start-services running" if ! pgrep -f
clipboard_daemon.py > /dev/null 2>&1; then nohup python ~/clipboard_daemon.py >
~/clipboard.log 2>&1 & fi if ! pgrep -f screenshot_watcher.sh > /dev/null 2>&1; then
nohup bash ~/scripts/screenshot_watcher.sh > ~/watcher.log 2>&1 & fi
```

Useful packages:

```
pkg install nmap openssh termux-api rclone pip install blake3 flask requests reportlab
```

Links:

GitHub: github.com/weissmann93/NeoBild

MNN: github.com/alibaba/MNN

Termux: f-droid.org (not Play Store)

CISA KEV: cisa.gov/known-exploited-vulnerabilities-catalog

MAXXKI Analyzer: github.com/maxxki/maxxki-analyser

Prototype Fund: prototypefund.de

Support This Project

NeoBild is free and open source. Every contribution goes directly into hardware, development time, and keeping this project independent from Big Tech.

```
BTC: bc1q0897wq7ze5500w7hypcme0czkqgwlxd3h6p9aj XMR: 47uoRpaykxzWHY1W7oRVeyX5w42GS9uA5Wv3Kp8iBpDHKqpJfxmVPyC5iQDCfT6B5z592fnYyX5YSKjxxwSfmksZ7XCCJti GitHub Sponsors: github.com/sponsors/weissmann93
```

Guide Version 2.0 — April 2026 · Written entirely on a Xiaomi Redmi Note 14 Pro+ · Snapdragon 7s Gen 3, Android 14, HyperOS